CS-368 Reinforcement Learning
Issue Date: November 18, 2025
Semester Project
Deadline: December 21, 2025

Objective

The primary goal of this project is to tackle a **complex engineering problem** by solving a real-life Atari video game from the Gymnasium library using reinforcement learning (RL). You will learn to design, implement, and analyze RL algorithms in the context of video games, focusing on performance metrics such as convergence, reward maximization, and hyperparameter sensitivity. The project must be done in a group of **2 students**.

Project Requirements

1. **Problem Selection**
   - ❖ Choose an Atari environment from the Gymnasium library. Examples include:
     - **Pong-v5:** This classic game involves two paddles and a ball.
     - **Breakout-v5:** In Breakout, the agent controls a paddle at the bottom of the screen and must break bricks by bouncing a ball off them.
     - **Freeway-v5:** The agent controls a chicken attempting to cross a busy highway without being hit by cars.
     - **Seaquest-v5:** The agent controls a submarine, shooting enemies and rescuing divers.
   - ❖ Clearly justify your choice of environment and its challenges.
2. **Algorithm Design and Implementation**
   - ❖ Analyze the problem and develop an RL algorithm to solve the selected environment. Options include:
     - **Value Function Approximation**: Function-approximated value-based learning.
     - **Deep Q-Network (DQN)**: Q-learning with neural networks.
     - **Double DQN (DDQN)**: Improved DQN with reduced overestimation bias.
     - **Policy Gradient Methods**: Stochastic policy optimization (e.g., REINFORCE).
     - **Actor-Critic Methods**: Hybrid methods (e.g., A2C, PPO, SAC, TD3 or DDPG).
   - ❖ **Note**: The algorithm must be implemented (using libraries like PyTorch or TensorFlow for neural networks is allowed, you can also use prebuilt RL libraries like Stable-Baselines).
3. **Performance Evaluation and Analysis**
   Perform a comprehensive analysis of your RL algorithm, including:
   - ❖ **Convergence Analysis**:
     - Plot and analyze learning curves (e.g., episodic returns, loss function values).
     - Discuss how quickly the algorithm converges and its stability.
   - ❖ **Reward Maximization**:
     - Compare the agent's performance to benchmarks (e.g., random actions).
   - ❖ **Hyperparameter Sensitivity**:

- Experiment with hyperparameters (e.g., learning rate) and their impact.

❖ **Robustness and Scalability**:
- Evaluate how well the algorithm generalizes to variations in the environment.

❖ **Comparison with Existing Solutions** (if applicable):
- Briefly discuss how your results compare to benchmarks from literature or the Gymnasium documentation.

4. **Report Submission**
   Submit a report (PDF format) that includes:
   ❖ **Introduction**: Background on the environment, problem statement, and the algorithm's rationale.
   ❖ **Methodology**: Detailed description of your algorithm design, pseudocode, training pipeline, and hyperparameter choices.
   ❖ **Results**: Include plots, tables, and visualizations showcasing the agent's performance and analysis.
   ❖ **Discussion**: Challenges encountered, insights on the results, and potential improvements.
   ❖ **Conclusion**: Key takeaways and lessons learned.
5. **Presentation**
   Each group will deliver a **10-minute presentation**, including:
   ❖ Overview of the chosen environment and algorithm.
   ❖ Explanation of the implementation, training process, and results.
   ❖ Discussion of challenges, key findings, and future directions.

Deliverables

1. **Code**
   - Submit a well-documented codebase in a zipped folder.
   - Include instructions for reproducing your results (e.g., README file).
2. **Report**
   - Submit a comprehensive report in PDF format (max 8 pages, excluding references and appendices).
3. **Presentation**
   - Provide your slides in PowerPoint or PDF format.

Grading Criteria (Rubrics)

Implementation and Report:

Criteria	Description	Weight	CLO
Environment Selection	Clearly explains the choice of Atari video game environment and its relevance as a complex problem. Discusses the unique challenges of the environment (e.g., control dynamics, state space).	10%	CLO-2

Criteria	Description	Weight	CLO
Algorithm Justification	Provides a strong rationale for selecting the chosen RL algorithm. Connects the algorithm's strengths to the specific challenges of the selected environment.	15%	CLO-2
Algorithm Implementation	Correctly implements the algorithm with clear and efficient code using Gymnasium and other libraries. Includes appropriate handling of the environment (e.g., reward shaping, normalization, etc.). Code is well documented.	30%	CLO-4
Convergence Analysis	Evaluates convergence behavior with clear metrics (e.g., episodic returns, loss values). Includes visualizations (e.g., learning curves) and insights on stability and speed.	15%	CLO-2
Reward Maximization	Demonstrates the ability of the agent to maximize rewards over episodes using visualizations and plots. Use of libraries such as matplotlib etc. Includes plots showing improvement over time and quantitative metrics comparing baseline performance.	20%	CLO-4
Hyperparameter Sensitivity	Conducts thorough experiments on key hyperparameters (e.g., learning rate) and analyzes their impact on performance with supporting plots.	10%	CLO-2

Presentation (Analysis and Design):

Criteria	Description	Weight	CLO
Environment Selection	Clearly explains the choice of Atari environment and its relevance as a complex problem. Discusses the unique challenges of the environment (e.g., control dynamics, state space).	15%	CLO-2
Algorithm Justification	Provides a strong rationale for selecting the chosen RL algorithm. Connects the algorithm's strengths to the specific challenges of the selected environment.	20%	CLO-2
Algorithm Implementation	Correctly design and implements the algorithm with clear and efficient code. Includes appropriate handling of the environment (e.g., reward shaping, normalization, etc.).	20%	CLO-3
Convergence Analysis	Evaluates convergence behavior with clear metrics (e.g., episodic returns, loss values). Includes visualizations (e.g., learning curves) and insights on stability and speed.	15%	CLO-2
Reward Maximization	Demonstrates the ability of the agent to maximize rewards over episodes. Includes plots showing improvement over time and quantitative metrics comparing baseline performance.	15%	CLO-3

Criteria	Description	Weight	CLO
Hyperparameter Sensitivity	Conducts thorough experiments on key hyperparameters (e.g., learning rate) and analyzes their impact on performance with supporting plots.	15%	CLO-3

Collaboration Policy

- Each group must collaborate effectively, ensuring equitable distribution of work.
- Provide a brief note in the report on each member's contributions.

Guidelines for Success

1. **Plan Early**:
   Start by selecting your environment and reviewing its dynamics. Allocate time for debugging and experimentation.
2. **Visualize Results**:
   Use tools like Matplotlib, Seaborn, or Plotly to create meaningful plots and charts.
3. **Document Everything**:
   Keep detailed records of experiments, including failed attempts and insights gained.
4. **Be Creative**:
   Think critically about the challenges and potential improvements in your solution.

Mapping of Attributes of CEP (WPs)

Complexity Attributes	Justification
WP-1 (Depth of Knowledge Required)	Solving an Atari environment with RL requires deep knowledge across multiple domains: reinforcement-learning theory (policy/value methods, exploration/exploitation), numerical optimization, neural-network architectures, signal preprocessing (frames, stacking, observation normalization), training infrastructure (experience replay, target networks, GPU usage), and experimental design (hypothesis testing, hyperparameter sweeps). • WK-3 (Engineering Fundamentals) • WK-4 (Specialist Knowledge) • WK-5 (Engineering Design) • WK-8 (Research Literature)
WP-2 (Range of Conflicting Requirements)	The project presents trade-offs and conflicting objectives: maximize episodic reward vs. stability of learning; sample efficiency vs. compute/time budget; model complexity vs. reproducibility (smaller models easier to reproduce); exploration aggressiveness vs. safe/effective policies. Implementation choices (e.g., DQN vs. actor-critic) require balancing these competing goals.

WP-3 (Depth of Analysis Required)	There is no single "correct" algorithm or hyperparameter set for a given Atari game. The task requires original analysis (convergence diagnostics, learning-curve interpretation, robustness tests, sensitivity studies) and possibly novel tweaks (reward shaping, network architecture, normalization) to achieve good performance. Convergence/stability issues commonly require creative diagnostic approaches.
WP-7 (Interdependence)	The project decomposes into many interdependent sub-problems: environment selection and justification; algorithm choice and theory; neural-network design; data pipeline (frame processing, replay buffers); training infrastructure (checkpoints, parallelism); experimental design (baselines, hyperparameter grid); analysis and reporting. Failures in one area (e.g., bad preprocessing or buggy replay buffer) cascade into training and evaluation, so the problem has strong interdependence.